

his information is meant to guide the direction, it's okay if it's not 100% perfect.

.. What's essential is that there's enough information to allow the workshop to progress. It's perfectly okay to mention that the team can return to this async later if necessary.

· Move on to the next activity

Step 2: Clustering theme inputs (45 minutes + more if necessary)

.. This is the most critical part of the workshop as generating themes relies on this activity to be done carefully. If teams have mutually exclusive and collectively exhaustive theme bundles, you'll know this was successful.

1. Set the scene for the activity

· Read the Step 2 instruction in FigJam out loud and ask if there are any questions.

"I've transferred the inputs you linked me to in the workshop issue. Now working together, we are going to group these stickies thematically based on the user need they are addressing"

.. In FigJam Start the timer for 45 minutes

2. Observe and keep participants on track

- Ensure participants are working together and not independently.
- Prompt participants when clusters are beginning to form:
 - Ask if any other inputs could fit into that cluster.
 - Ask if any JTBD, Deferred UX, UX Bugs, or Actionable Insights relate to the cluster.
 - Inquire about clusters that are formed and prompt participants to answer questions about the problem it is solving.

As clusters start to form, you or the co-facilitator should copy and paste the Beneficiary, Job, Outcome text blocks below each bundle (see this steps instructions section near the bottom) so teams can begin filling out that information.

.. Watch out for large clusters. Also, ensure the participants are keeping the scope in mind. (Design work can be completed within (1-3) milestones, including solution validation).

.. Watch out for inputs within clusters that aren't solving the same problem.

.. If teams are struggling, propose using the job map template as a guide for the exercise.

.. The team may find it helpful to add a name/title above the group that helps explain what that grouping represents.

3. Wrap-up the activity

A team will rarely complete this activity within 45 minutes. Once the time is up, assess the team's progress and plan for a 5 minute break.

>· To reiterate, this is a vital step in the process, and you should not move on without defining clusters.

- Take the planned 5 minute break

Break (5 minutes)

This can be an intense process, and participant engagement will fade as the session continues. Including a break is critical for participants to remain focused and engaged in the session.

- Move on to the next activity or continue the prior activity if incomplete.

Step 3: Refining clusters into Themes (45 minutes)

1. Set the scene for the activity

- Read the Step 3 instruction in FigJam out loud and ask if there are any questions.
- In FigJam Start the timer for 45 minutes or however much time remains in the 2 hour workshop, ensuring to reserve at least 5 minutes at the end for a recap and next step guidance.

2. Observe and keep participants on track

Theme statement: DO THIS LAST for every theme. You will need all the other info before constructing the theme statement.

· To help with theme statements, Ask, "If I know nothing about this product area, would I be able to tell what this theme is addressing at a glance by reading this?"

Business objective: ensure teams don't get caught up in this aspect. This can be filled in after the Theme issue is created; however, it is still an essential part of the process.

Confidence: Use the chart in FigJam to have teams assess their confidence. Remember, we do not want to start design work on themes with low confidence.

3. Wrap-up the activity

Have the team look over the themes once they are done.

Ask:

- Are these mutually exclusive?
- Are these collectively exhaustive as they relate to the goals & vision (See Step 1)?
- Are these appropriately scoped (i.e. can be completed in 1-3 milestones)?

Conclude

Briefly summarize the activities in the workshop, and celebrate the team's progress. Something

to the effect of:

"Today we created goal and vision statements, bundled work into themes, refined themes and generated the inputs to prioritize them in your roadmap!"

- Ask if there are any questions about the next steps and end the meeting.

Post workshop

Following up

We advise following up with the team in the workshop issue a few days after the workshop has concluded reminding everyone of the next steps. We want to ensure the activities and concepts are fresh so teams won't have to recall every instruction from the workshop; The more time passes, the harder it will be for a team to complete their themes.

SECTION: product/ux/product-designer/_index.md

Working on issues

This section outlines our approach to handling issues. It's crucial to communicate with your Product Manager (PM) and Engineering Managers (EM) early and frequently to determine the best way to collaborate. Each team may have unique processes tailored to their specific product area and team needs.

Triaging UX issues

Every Product Designer is empowered to triage issues labeled with "~UX", "~Deferred UX" and "~UI polish". If you are not the one triaging, you should be included for feedback by the responsible PM and EM. Use Priority labels to suggest when the issue should be resolved and Severity labels to indicate its user impact. Always coordinate with your PM and EMs on the assigned labels.

Scheduling issues in a milestone

All issues labeled Deliverable that require UX will be assigned to a Product Designer by the kickoff. Issues labeled Stretch may or may not be assigned by the kickoff. Learn more about how we use Workflow labels in the GitLab Docs.

Communicating scheduled UX issues to the stage group

Consider adding a User Experience section to your team's planning issues (example 1, example 2). This section can include active design items for the milestone, such as research projects, Deferred UX, UI polish, or Pajamas components. Learn more about planning and capacity for Product Designers.

Including design problems in the planning issue helps make design efforts visible to the team and encourages cross-functional collaboration during workflow::problem validation,

workflow::design and workflow::solution validation.

Key benefits of making User Experience an official part of group's milestone planning include:

- Advocating for UX within the stage group by making it part of the monthly planning discussions.
- Using the planning issue with the Product Manager during the milestone kickoff recording.
- Communicating current design and research efforts regularly to customers and counterparts outside the stage group area.
- Sharing the Product Designer's capacity with the team.
- Encouraging early collaboration with counterparts during the design phase.

Product design process

Define the opportunity

- Collaborate with your PM to validate who you're designing for, what you're designing, and why you're designing it.
- Help your PM articulate the who/what/why as a user story. For example, "As a (who), I want (what), so I can (why/value)." If you're asked to implement a non-evidence-based how, encourage the PM to refocus on the who/what/why to collaboratively determine the best how.
- Assist your PM to defining MVC success criteria, prioritizing MVC ·must-haves· and non-MVC ·should-haves· and ·could-haves·. Note that these criteria may change based on new insights from the iterative design process and customer feedback.

Before you design

Generate ideas

As a Product Designer, a key part of your role is to lead and facilitate idea generation within our teams. While it's important to address known UX problems and work on product roadmaps with PMs, remember that there are also undiscovered issues worth exploring. Here are some activities and resources to inspire your creativity:

- Host workshops: Organize synchronous (e.g., ThinkBig!) or asynchronous workshops to brainstorm ideas. Define a scope and invite participants from Product, Engineering, UX Research, and other areas for best results.
- Engage with counterparts: Reach out to sales, customer success, or marketing counterparts for fresh perspectives. Consider inviting them as optional attendees to your regular meetings.
- Conduct problem validation research: Collaborate with Product Managers and UX Researchers to prioritize a round of problem validation research. Engage with customers to understand their challenges, what disrupts their productivity, and what keeps them up at night.
- Discover unknown pain points:
 - Use Dovetail to analyze data, collaborate on insights, and access our current research repository.
 - Use Chorus.ai to search and listen to sales call recordings and transcriptions for insights.
 - Explore Zendesk to identify existing problems, though it may require effort to parse through tickets due to non-optimal categorization for UX purposes.

For access instructions to Dovetail, Zendesk and Chorus.ai, refer to the relevant guides.

- The community forum is a support option for free users.

Understand the space

- Review existing research: Investigate the UX Research Archive, Dovetail, or other data sources to inform your decisions and measure results. If no relevant UX research exists, contact your UX Researcher to conduct or guide you and your PM in conducting research.
- Conduct competitive analysis: Analyze competitors to understand terminology, functionality, and UX conventions. Adhere to industry standards unless there is a strategic reason for deviation, such as leveraging disruptive innovation opportunities. Familiar conventions can ease user migration from other tools to ours.
- Create user flows and journey maps: Develop user flows or journey maps to ensure a comprehensive understanding of the workflow, and effectively communicate this to your team.

Investigate possible dependencies

Product Designers need to assess how their work impacts other parts of the product and the efforts of fellow Product Designers.

- Collaborate with peers: Proactively reach out to other Product Designers via Slack, UX Sessions, etc., to gather background information and understand how your work interacts with other product areas.
- Identify and involve the DRI: Determine the Directly Responsible Individual (DRI) for the product area you are working on and involve them from the start. If unsure, consult the Product Categories Handbook page.
- Review product kickoff: Check the Product Kickoff Review to see the list of issues planned for next release in other stages.

Designing with AI

When designing AI solutions, Product Designers need to approach the process thoughtfully to ensure that the technology enhances the user experience and meets strategic objectives. Follow this guide to support your work: [Designing with AI: A Guide for Product Designers](#)

Aiming towards "sophisticated simplicity"

The visual design of GitLab has evolved from naive simplicity to sophisticated complexity. However, the ultimate goal is to achieve sophisticated simplicity (Sophisticated Simplicity, Marcel Weiher). When designing, keep these three principles in mind:

1. Structure - Organize and arrange content and concepts into meaningful groups and patterns.
1. Discovery - Ensure users can interact and explore in ways that promote learning and proficiency while minimizing mistakes.
1. Capability - Provide features and functions that enable users to complete tasks and automate processes.

Balancing these principles is crucial for an optimal user experience:

- Structure + discovery without capability: Results in a simple experience suitable for static content but lacking functional richness.
- Discovery + capability without structure: Creates a robust and intriguing experience that is difficult to master due to a lack of guiding structure.
- Structure + capability without discovery: Leads to sophisticated complexity functionality is abundant, but users struggle with learning, association, and understanding cause and effect.

Achieving sophisticated simplicity reduces friction for accessing basic functionality, provides quick access to powerful features, and helps users become proficient in completing tasks.

Consider these questions when designing:

- Is the content hierarchy and flow clear?
- Are similar items grouped, and are the groups clearly defined?
- Does this content or functionality need to be visible all the time and for everyone?
- Is this content or functionality necessary in this context? Is it both helpful and essential?
- Does discovery help users avoid mistakes or recover from mistakes easily?
- Does the structure support discovery and the use of advanced capabilities?
- Is this feature or capability needed or used, or what would happen if it were removed?
- Is everything "in reach," or are users left wandering?

"Unboxing" the UI

Ongoing design efforts align with sophisticated simplicity:

1. Unboxing the UI: Reduce the overuse of boxes for conveying hierarchy, organizing content, and dividing pages into sections. Explore better alternatives in this video on "Unboxing" the UI.

Ideate and iterate

Iterative design at GitLab combines "incremental design" and "design iteration." It involves breaking down design solutions into the smallest changes that improve user outcomes. The goal is to quickly integrate these changes into the product to gather early feedback and guide refinement.

When applying iterative design, consider the longer-term strategy and collaborate with your Product Manager to plan successive releases until the vision is realized.

- Share design ideas: Start with the lowest fidelity that communicates your idea. Increase fidelity as design confidence grows and implementation approaches.
- Request feedback early: Regularly seek feedback from your PM to refine your understanding of the problem and solution criteria.
- Engage engineering peers: Involve engineers early to get insights on technical feasibility and costs, and invite their design feedback.
- Design reviews: Participate in Design Reviews with other Product Designers for objective feedback and new ideas, helping to improve your work and gain additional context.
- Collaborate with Technical Writers: For substantial UI text changes, work closely with your

group's Technical Writer. For smaller changes, involve them in the review process.

- Involve your Product Design Manager: For significant UX changes, include your Product Design Manager in feedback sessions for broader insights and alignment with other initiatives.
- Validate with users: If high-risk or uncertain directions, validate solutions with users through UX research. For lower-risk directions, gather feedback from users after releasing the minimal valuable change (MVC).
- Use the design and UI changes checklist: Refer to the design and UI changes checklist to ensure your design's readability, appearance, and functionality.

GitLab Design Talks: Iteration

{{< youtube "0lhjzU-QZ2w?start=286&end=359" >}}

> "Our relationship with uncertainty: When we conduct research and design we have some level of certainty about how effective it's going to be, but it isn't until we ship it and get it in the hands of many users that we truly understand how effective the thing is that we designed."

{{< youtube "VrXQiiK3Q9U?start=244&end=334" >}}

> "Breaking things down creates psychological safety for me as a designer."

Design reviews

Sharing work and gathering feedback can happen at any stage of the design process, often through mocks and open discussions in issues. Design Reviews are dedicated sessions for Product Designers to give and receive specific feedback, with benefits such as:

- Discovering what others are working on.
- Identifying overlapping work.
- Surfacing opportunities for group collaboration.
- Encouraging the practice of sharing work.

We prioritize asynchronous design reviews to allow broader participation. Here's how to conduct an asynchronous review:

1. Identify key issues: Choose an issue with many open questions or one you're most excited to work on.
1. Select the best format: Share your work using an issue, text blurb, screenshots, Figma file, or a short video walkthrough.
1. Provide context: Share the customer problem, constraints, and specific feedback needs. Clearly state what feedback you want and where to provide it.
1. Post for feedback: Share your work in the ux-coworking Slack channel and relevant group channels, providing links to the item and the feedback location.
1. Record videos: If recording a video, use Zoom and upload it to the GitLab Unfiltered YouTube channel. Set the visibility to "Public" unless confidential and add it to the "UX" playlist.
1. Comment with references: Comment on your feature issue with a link to the video and related references (example), such as issues, epics, and Figma files. Add these links to the video description.
1. Capture feedback in issues: Open an issue dedicated to capturing feedback (example), attaching all necessary references and information needed for review.

Examples

- Add a new list iteration - feedback request
- Swimlane boards loading states - Skeleton loaders feedback request
- Right issuable sidebar patterns feedback request
- Geo: Maintenance mode design

Who to include in design reviews

Determining who to include in a design review can be challenging. Here are some guidelines:

- Stage group: Your Product Manager, Engineering Manager, Frontend Engineers, and Product Design Manager should always have opportunities to review and provide feedback. Include them in the issue for ongoing collaboration.
- Designers: Engage peer designers within your Section at any phase of the design process.
- Cross-stage counterparts: If your work impacts other Stages, include those counterparts.
- Broad Impact counterparts: For changes to navigation, global headers/footers, Pajamas patterns, request a UX Department review in the uxcoworking Slack channel or mention @gitlab-com/gitlab-ux/designers in GitLab.
- Other departments: If your work involves other departments (e.g., Customer Success, Sales, Marketing), invite them to provide feedback, especially when deviating from brand guidelines.

If unsure who to include, consult your Product Design Manager for guidance.

Design critiques

Design critiques are dedicated sessions where Product Designers exchange rigorous, constructive feedback to elevate work quality; challenging assumptions and examining the underlying rationales for design decisions beyond standard design reviews. Benefits include:

- Pushing designs to excellence through specific, actionable feedback that increases overall quality and value for customers
- Building a culture where constructive challenge is valued and expected
- Accelerating professional growth and design craft through deeper discussions
- Addressing issues and inconsistency earlier in the design process
- Alignment with the design system, other product areas, and UX paradigms
- Ensuring user-centered thinking remains at the forefront

We prioritize a balanced approach to design critiques that practices benevolent, radical candor and cares personally, but challenges directly. Here's how to conduct an effective critique:

1. Prepare the work: Select designs that would benefit from rigorous feedback, whether early concepts or refined work.

1. Structure the critique session:

1. Provide thorough context: Share the customer problem, constraints, design decisions made, and areas where you specifically need critique. Be clear which aspects you want challenged. (5 minutes)

1. Begin by presenting work without excessive explanation. (10 minutes)

1. Allow participants to ask clarifying questions and provide critique. (bulk of the sessions time)

1. Summarize key takeaways and next steps. (5-10 minutes)
1. Participate effectively:
 1. Be curious by asking questions like "have you·," "how does this·," and "why is this·" to help you understand the design decisions and solutions.
 1. Provide specific and detailed comments that help encourage, challenge, and unblock the presenter.
 1. Connect feedback to user needs and business goals instead of personal preference.
1. Receive critique productively:
 1. Listen openly before responding.
 1. Take detailed notes and ask clarifying questions when needed.
 1. Thank participants for specific, challenging feedback.
1. Document and follow up:
 1. Capture key critique points in the relevant issue or epic.
 1. Share how the critique influenced your design decisions in subsequent iterations.
 1. Acknowledge team members whose critique led to meaningful improvements.

Setting the right mindset

To ensure critique sessions maintain both psychological safety and honest feedback, facilitators are strongly encouraged to reference the code of conduct and verbalize the mantra before beginning.

Code of conduct

During our critique sessions, we commit to:

1. Focus on the design work, not the designer.
1. Communicate with respect and avoid offensive language or behavior.
1. Challenge directly while showing we care personally.
1. Provide specific, actionable feedback and reasoning rather than vague comments.
1. Practice genuine curiosity to understand design decisions before critiquing them.
1. Remain open to different perspectives and approaches.
1. Receive critique openly and be willing to let go of unsuccessful elements.
1. Engage in candid conversation and spirited debate with passion for our craft.

Mantra

Critique the work, not the person.

Care personally, challenge directly.

Speak candidly, listen openly.

Embrace curiosity, let go of attachment.

Note: Reading the mantra verbatim before each session might initially feel awkward or repetitive, however, consistently expressing these shared expectations creates the right environment for productive critique. The few moments it takes pays dividends in the quality of feedback and team dynamics.

Partnering with Technical Writers

When adding or changing to UI text, it's essential to collaborate with your group's Technical

Writer. This collaboration should begin during the Product Design phase.

UI text includes button or menu labels, error messages, log files, user-assistance microcopy, notification emails, and any other text visible in the UI. Changes to UI text can significantly impact documentation steps.

To ensure a smooth process:

- Label the issue and MR: Apply the UI text and documentation labels.
- Request a review: Message the Technical Writer for the group in the design issue to request a review. Specify files or lines to review and how to preview or understand the context from the user's perspective.
- Collaborate on finalizing text: Work closely with the Technical Writer to finalize the UI text, ensuring it's usable and accurate.
- Incorporate feedback from other departments: When other departments (e.g., Product Management, Marketing, Legal) provide input on UI copy, ask for goals rather than specific text to avoid extensive revisions and design-by-committee.

Collaborating on in-product reference information

Sometimes the designer, PM, and technical writer agree to display additional in-product reference information in a drawer component. The reference information should align with the existing documentation for the feature.

1. Draft copy: The designer drafts the content for the drawer, identifying necessary information for user goals. This draft should be as close as possible to the final version.
 1. Mark draft status: Indicate that the drawer content is not final, using a text watermark like "waiting on documentation," "draft," or "placeholder," or a pin in the Figma file.
1. Follow design to development process:
 1. Developers create the drawer and documentation as part of feature code MR.
 1. The Technical Writer reviews the documentation and drawer content.
 1. Once the documentation is published, the drawer is populated with the content.

Partnering with UX Researchers

UX Researchers collaborate with Product Managers and Product Designers to ensure research projects are targeted and provide valuable insights.

<!-- - Request research: Follow the process to request research, even if conducting the research yourself.-->

- Document findings: Ensure research findings are documented according to established processes.

Refine MVC

To maintain focus and avoid scope creep in UX issues:

- Prioritize must-haves: Work with your PM and developers to identify "must-have" versus elements that can be deferred. Document non-MVC concepts and research in new issues, linking them to the original issue. Consult your Product Design Manager if you need help breaking down

large issues.

- High confidence, low risk changes: If developers need to start before designers are validated, focus on high confidence, low risk changes while the rest of the solution is validated.
- Plan ahead: PMs and UXers should work together to get 1-2 months ahead, ensuring the Build track always has well-validated product opportunities ready to start.
- Manage large features: Features should be buildable within 1-2 milestones. If too large, work with your PM and Engineering team to split the feature into smaller, manageable segments.

For inspiration, watch our Product Designers discuss iteration at GitLab.

Present an MVC solution

- Propose one solution:
 - Once you've validated your solution with users, propose a single solution. Suggesting multiple alternatives can undermine your expertise as a UX professional and lead to design-by-committee. If you must propose multiple solutions, clearly explain your reasoning.
- Share context and goals asynchronously:
 - Provide all necessary context in your issue to ensure your audience understands your proposal and knows how to assist. Clarify who the solution is for, what it will enable them to do, and whether you need feedback, assistance, or approval. Highlight changes since the last review to facilitate understanding.
 - Use collapsed content sections to include supportive information without distracting from the main point. Refer to this issue comment for an example.
- Request feedback from your Product Design Manager:
 - @mention your Product Design Manager for feedback. They can provide strategic alignment, ensure quality, and maintain functional consistency across the product.
- Focus on the customer and problem:
 - Frame design discussions around the customer and the problem being solved, not the UI or functionality. Begin with the current state, explain how it fails to meet user needs, and present the proposed solution from the user's perspective. Continuously relate everything back to the user's experience.
- Anticipate and address questions:
 - Anticipate potential questions and address them in your proposal comments. Explain your rationale to reduce feedback loops and unnecessary discussions. This is especially important when proposing changes to established patterns.
 - Use questions as headings:
 - Structure your proposal with headings framed as questions to provide clarity and focus.
- Keep the SSOT updated:
 - Ensure the Single Source of Truth (SSOT) is updated with all agreed-upon elements, including images or links to design work.
- Follow design file guidelines:
 - Regularly commit your design files following the GitLab Design project contribution guidelines.
- Consider new UX paradigms carefully:
 - Evaluate if the new pattern will be inconsistent with other areas, if other areas need updating, and if the new pattern significantly improves the user experience. If changes are necessary, follow the component lifecycle documentation.

Delivering your solution

- Update the issue description:
 - Once your work is complete and feedback is addressed, update the issue description (the SSOT) with a "Solution" section. This should be the reference point for what should be done and how.
- Include your design:
 - Add your design to the "Solution" section. For small designs, a mock-up may suffice. For more detailed changes, include a link to the Figma file.
- Use the design handbook checklist:
 - Follow the design handoff checklist to make sure all design specifications are documented and engineers are set up for success.
- Leverage collaboration tools:
 - Utilize both Figma's collaboration tools and GitLab's design management features. Refer to the following table for common scenarios and recommended tools. Use your judgment to decide the best way to move the design forward.

Scenario	Figma	Design Management
-----	----	-----
Co-designing within a shared file	·	·
Providing, or seeking feedback while a design is still in progress, and not ready for MVC	·	·
Seeking feedback on a design with a larger audience	·	·
When feedback directly impacts an issue	·	·
Presenting design options or variations so the team can choose a direction	·	·
Sharing a prototype	·	·
Adding a to-do for a designer as it relates to in-progress design	·	·
Adding a to-do for a designer as it relates to an issue	·	·
Identifying visual regressions	·	·
Detailed redlines or specs	·	·

- Break down implementation issues:
 - If the solution needs to be broken out into smaller issues for implementation, apply the workflow::planning breakdown label. Stay involved by walking PM and Engineering through the proposed solution and participating in the conversation to break down the issue.
- Schedule with PM and EM:
 - If the solution needs to be scheduled by PM and/or EM, apply the workflow::scheduling label and mention the responsible product manager to schedule it. Communicate with the assigned engineer to ensure they understand the solution.
- Review the current milestone implementation:
 - If the issue is meant for implementation in the current milestone, review the solution with the assigned engineer(s) and/or EM. If they are comfortable with the solution, apply the workflow::ready for development label.
- Handling premature moves to Build phase:
 - Sometimes, a Product Manager might request moving an issue to the Build phase before it meets UX Department standards. In such cases create follow-on issues and/or apply the Deferred UX label to indicate that the product doesn't meet UX requirements and will need immediate iteration.

Socialize your work

Sharing design work allows Product Designers to mentor, engage, and inspire peers inside and

outside GitLab. It widens your perspective and helps you understand the broader impact of design decisions on the product.

Internal Socialization:

- Slack: Share insights and updates.
- Unfiltered YouTube: Provide updates and gather feedback.
- UX Forums: Open pathways for collaboration with teams addressing similar objectives and JTBDs.

External Socialization:

- Blog posts: Write about your design decisions and processes.
- Speaking at events/conferences. Share your work and insights.

These activities not only inspire other designers but also increase visibility for GitLab's UX Department, enhancing transparency and attracting potential applicants.

Follow through

- Scope down features:
 - Encourage engineers to break features into multiple merge requests (MRs) for easier, more efficient reviews.
 - Consider the UX impact of merging partial changes. Use feature branches or flags if partial changes negatively affect the UX to ensure the full scope ships together.
- Communicate end goals:
 - When breaking solutions into smaller parts, make sure the entire team understands the end design goal to align development efforts.
- Maintain SSOT:
 - Keep the issue description updated with the agreed scope, maintaining it as the Single Source of Truth (SSOT). Remove or archive outdated content.
 - Subscribe to and regularly review issues in your product area. Actively contribute to planning meetings to ensure proper prioritization.
- Engage in issue resolution:
 - Stay assigned and subscribed to active issues. Follow related MRs and address any additional UX issues that arise.
 - For obvious changes, update the SSOT directly without waiting for consensus, using your judgment.

MR reviews

Refer to the MR reviews handbook page.

- When reviewing an MR, refer to the SSOT in the issue, which should include a mock-up or Figma link in the "Solutions" section of the issue description.

Follow-up after a design proposal is finalised

For changes that affect Pajamas (GitLab's design system):

- New UI component: Create a UX Pattern issue in GitLab Design and follow the checklist.
- Documentation updates: Create an issue and/or MR in the Design System.
- Application updates: Create issues to update affected areas of the application.
- Socializing changes: Add an agenda item to the UX Call to inform the team of changes.

Follow-up after your changes go live

- Gather feedback:
 - Listen for feedback from social media, Slack, forums, and internal/external sources.
 - Create an issue to track and address the feedback.

By following these steps, you ensure effective communication, collaboration, and continuous improvement in the design and development process.

SECTION: product/ux/product-designer/ai-design.md

Understanding AI Design Challenges

The main difference between designing AI features and traditional features is uncertainty. With Large Language Models (LLMs), we face uncertainty in both:

What users will do or write
How the LLM will respond

As designers, we have less control over user inputs and system outputs than we're used to. This guide provides tools and methods for working in this uncertain environment, supplementing your existing design toolkit.

Key Questions This Guide Addresses

- How to map user happy paths with unpredictable inputs and outputs
- How to conduct usability testing with variable sessions
- How to identify unhappy paths and potential failures
- How to measure task success with subjective LLM responses
- How to design for unpredictable conversational flows

Design Process for AI Features

Before you design

Define the opportunity

During problem validation, you should work closely with Product Managers to understand who you're designing for, what you're designing, and why you're designing it.

For GenAI, the problem and solution space may be broader than usual. For example, when working on something like Duo Workflow or Duo Chat, the team may be tempted to say "the feature can assist with anything the user asks". This approach will make it difficult to evaluate your

solution later. You also run a risk of building a solution that does many things poorly, and nothing really well.

Instead, use the following page to help you:

- Scope the problem space clearly
- Define what problems your solution will and won't solve
- Set measurable success criteria

AI Engagement Guidelines

We recommend that designers create AI Engagement Guidelines before moving into the design phase. This is a framework proposed by People + AI Research at Google that we propose to use here. This framework provides a foundation for establishing alignment on AI related design and technical requirements. We have adapted the framework into our own Figjam template [{tbd link}](#).

Resource: Interaction Design Policies: Design for the opportunity, not just the task..

By creating AI Engagement Guidelines, you define the target behavior (UX) of the model. Through discussions with your PM and dev team, you should clearly define what you expect the model to do, what it shouldn't do, and where the system or user might fail. By establishing AI guidelines, you ensure stakeholders are aligned on model behavior and expected outputs.

Map the user flow

User flow design for AI focuses not on predicting outputs but on creating consistent ways for users to interact with, shape, and recover from situations when AI fails them. When creating user flows, you are defining the happy path and unhappy paths.

A happy path in generative AI design is when users can successfully achieve their goals through clear input, appropriate AI outputs, and minimal corrections, even though the exact inputs and outputs may vary with each interaction.

Happy path example:

- User begins: "Analyze this long document for key insights"
- System tracks request count and shows clear usage meter
- Warning appears at 75% with "Save and Continue Later" option
- Progress auto-saves with completion timestamp
- Analysis resumes next day from saved point

An unhappy path occurs when AI interactions require user intervention to correct or recover from unexpected outputs.

Unhappy path example:

- User begins: "Analyze this long document for key insights"
- System fails after hitting undisclosed rate limit
- Shows generic error: "Something went wrong"
- User loses all progress upon refresh
- User has to start over

Key failure points:

- No visibility into system limits
- Unclear error messaging
- Loss of user progress
- Forced restart of workflow

Since these moments are frequent in generative AI interactions due to output variability, designers should treat them as core flows rather than edge cases. Example of unhappy paths:

- Technical failures
- AI returns incorrect or incomplete responses
- System errors or timeouts
- Token limit issues
- Performance degradation
- User Behavior variations
- Unexpected queries
 - Context switching mid-conversation (if designing for conversations)
 - Misunderstanding of AI capabilities (i.e. a user thinks Duo knows their codebase)
 - Complex queries that combine multiple intents (if designing for conversations)
 - Requests outside system capabilities (i.e. a user thinks Duo knows their codebase)

We're not designing for what the AI will say - we're designing for how users will work with whatever it says. These patterns are predictable and designable, even when the outputs aren't.

User flow design for generative AI looks like this:

- Map ideal flow first
 - Define the best-case scenario step by step
 - Include both user actions and system responses
 - Identify key success points
- Identify likely breakdown points
 - Where might the AI misunderstand?
 - When might system limits be hit?
 - What user expectations might not be met?
- Design recovery paths
 - How users get back on track
 - Clear error explanations
 - Alternative paths to success
- Build in feedback mechanisms
 - How users signal problems
 - Where system shows understanding
 - Ways to correct or refine

Define the ideal output

While generative AI outputs are non-deterministic (producing different results each time), defining ideal outputs is crucial for both prompt engineering and interaction design. With clear definitions of an ideal output, we streamline the UX for the user between generations. These definitions also shape how we instruct the model and design user interactions in the prompt.

Key elements to define

- Output structure
 - Length (character limits, paragraph count)
 - Format (JSON, markdown, plain text)
 - Structure (headers, sections, bullet points)
 - Style (formal, casual, technical)
- Content requirements
 - Essential information (core data points, key metrics)
 - Required data points, if any (technical specs, citations)
 - Level of detail required (summary vs in-depth)
 - Accuracy needs (fact-checking, validation)

Example: You're building an automated API documentation generator

- Length: Each endpoint described in <500 chars
- Format: OpenAPI specification in YAML
- Structure: Standardized sections for endpoints, methods, parameters
- Style: Technical reference format with standardized terminology

While we can't predict exact outputs, defining these elements helps create consistent user experiences and better prompt engineering. Experiment with creating your ideal output using Claude, LangSmith, or locally. [{link to docs}](#)

LLM Judge Evaluation

Additionally, begin to discuss with your team how you'll evaluate the answers provided by the LLM. The team will eventually need a data set to use for LLM evaluations. It's best to work on the data set early, as it will force you to consider the types of prompts you expect to provide the LLM (or for users to provide the LLM), along with acceptable answers. If the team cannot confidently produce at least a preliminary data set, then user research or other data mining can be used to help with this step. This step is a cross-functional step, ideally with input from all team members.

During design

When designing the screen interactions, refer to GitLab Duo patterns in Pajamas.

There are many other great interaction design resources for AI. One we like is Shape of AI.

During solution validation

As with any project, you'll want to ask yourself how confident you are in your solution and if you aren't very confident, validate your solution with users before committing to development. You will want to select a validation method that works best for your research questions.

Generative AI solution validation has two distinct testing possibilities:

- low-fidelity or static Figma prototypes (highly moderated)
- interactive LLM prototypes (preferably unmoderated)

If you are using a low-fidelity (Figma) prototype

Most solution validation will be in the form of low-fidelity prototypes until prototyping tooling is built out. This should occur earlier in the process.

Starting with moderated testing gives you the ability to understand where the user will interact with the solution in the context of your workflow. It will also help you test assumptions about how you expect them to interact with the LLM. Mock up the solution in a Figma prototype that shows the happy path (and predetermined outputs).

During your session, aim to understand the user's current workflow before showing any designs. Once you understand their workflow, show them a Figma prototype for the solution.

A reminder that LLMs are interactive tools controlled by language and offer multiple paths for non-linear engagement. A Figma prototype is linear and will not offer the range of possibilities. You should be prepared to show multiple mock outputs that are an ideal of your expected generated response.

Since Figma prototypes are not generating responses to user queries, you'll need to have participants think out loud about their intended queries and expected responses before showing them the results from a button or predetermined output. After showing an output, spend time discussing the output and the reaction to it to determine if it is meeting expectations.

For example, instead of saying "Let's assume you write summarize issue" and asking for feedback on a result, keep the questions open ended and user led. Ask the participant "What would you type in this field?" "Why?" Then, show them a result. Then ask for feedback on your mock output. "What do you think of this response?" and "What would you expect from what you typed?"

Throughout the conversation, carefully observe where users desire control, express trust concerns, or want to edit AI outputs, as these are critical friction points in AI interfaces.

Always conclude sessions by asking about likelihood of adoption and potential dealbreakers, as users might provide positive feedback throughout but reveal critical adoption barriers in their final assessment.

These sessions will likely result in tweaks to the prompt to improve outputs and better meet user behaviors.

If you are validating a solution and you have an interactive LLM prototype

or feature that users can test live and provide feedback, unmoderated testing is ideal. Define the task you'd like the user to complete and always record the session.

During unmoderated sessions, pay attention to:

- User mental models
 - How users frame their requests
 - When they expect AI to understand context
 - Where they assume capabilities that don't exist
 - How they recover from misunderstandings

- Interaction patterns
 - Natural vs. prompted inputs
 - Response to different AI output styles
 - When users abandon and restart
 - How they handle variable outputs
- Trust signals
 - Points where users verify information
 - When they question AI responses
 - How they build confidence in the system
 - What causes trust breakdown
- Task completion
 - Whether users achieve their goals
 - If they find alternate paths to success
 - When they need human intervention
 - How they combine AI with other tools
- Recovery behaviors
 - What users do when AI fails
 - How they rephrase or retry
 - When they give up entirely
 - Ways they work around limitations

Guidance for MR Reviews or UX Scorecards

There are many times you'll test your workflows live, whether you are doing an MR Review or a UX Scorecard

UX testing ensures designers identify any potential issues that evaluation might miss. The focus on UX testing isn't on the model's accuracy but on how well the model and interface handles different inputs, failures, and edge cases.

- UX testing
 - Focuses on how to interact with the interface
 - Tests specific features, workflows, happy paths, unhappy paths
 - Identifies usability issues and friction points
 - Helps refine interaction patterns
- LLM Evaluation
 - Assesses AI system performance and outputs
 - Measures accuracy and usefulness of responses
 - Checks for biases and errors
 - Validates against business requirements
 - Helps improve model behavior

UX testing helps stakeholders better understanding user behaviors, strategies, and mental models for navigating AI interactions.

For example, where the LLM Evaluation might measure if an AI properly refactored a function to be more efficient, UX testing examines how developers iteratively describe their refactoring goals, interpret suggested code changes, handle merge conflicts, and learn to trust or verify the AI's recommendations.

To to UX testing for a new generative AI feature, return to the data set created at the beginning of the project. You will use that data set to test the performnace of the solution. If you are testing a conversational feature, you'll need to ask followup questions related to the data set to ensure the model retains proper context and answers properly.

UX testing should include testing happy paths and unhappy paths.

- Try various inputs from the data set
- Map happy and unhappy paths
- Document usability issues
- Identify friction points

Test edge cases:

- Empty or minimal inputs
- Extremely long inputs
- Special characters/formatting
- Rate limit boundaries
- System errors/timeouts
- Multiple rapid requests
- Context switching (for conversational solutions)
- Incomplete queries

Example of UX testing flow with dataset:

- Input test prompt
- Review AI response
- Note interaction patterns
- Ask followup questions if testing a conversational solution
- Test error scenarios
- Document recovery paths

Links and Resources

- UX Forum presentation on this framework
- AI-human interaction in Pajamas: Documentation on best practices for AI-human interaction.
- AI Integration Effort FAQ: Internal handbook with frequently asked questions about AI integration efforts. Internal handbook ·
- UX maturity requirements: Documentation on the UX maturity requirements to move AI features from Experiment to Beta to Generally Available (GA).
- Experiment, Beta, and Generally Available features: Guidelines on the different stages of feature availability.
- UX research in the AI space: Documentation on conducting UX research in the AI domain.

SECTION: product/ux/product-designer/capacity-management/_index.md

Planning and managing capacity

Product Designers are assigned to work within their stage group and take on responsibilities across the broader UX department. The following guidelines can assist in effectively managing responsibilities:

Product Designers

- Ensure discipline peers know about UX team assignments, such as department-level initiatives, and what you'll need from them to succeed.
- Account for time off (yours and others') during milestone planning.
- Inform your manager immediately if you think you won't complete your work on time. Early notification increases the likelihood of resolving the issue and managing expectations.
- Optionally, use UX issue weights to better understand your capacity and facilitate conversations with your Product Manager.

Product Design Managers

- If requested, help Product Designers set a baseline capacity for stage- and UX-team-assigned work each milestone.
- Quantify strategy work with clear time-to-complete (TTC) expectations, measurable goals, and due dates.
- Resolve team questions, concerns, and blockers quickly.
- Ensure cross-functional partners are aware potential dependencies.

Priorities

Designers are responsible for managing their own capacity and are a manager of one. The following categories help prioritize work throughout a milestone:

Must do:

- Merge request (MR) reviews, including community contributions. See MR Reviews guidelines.
- Feedback requests from other designers (issues, MRs, Figma, ux-coworking Slack channel).
- Adjustments to issues in the current release milestone and labeled Deliverable.
- Issues assigned with labels workflow::problem validation, workflow::solution validation, or workflow::design.
- Planning and prioritization of UX issues in the next release milestone. See Milestone planning.
- Sharing work with the team through UX Forums, Slack posts, UX meetings.

Should do:

- Tasks that improve understanding of users and their workflows (e.g. UX Scorecards).
- Issues in the current release milestone and labeled Stretch.
- Issues labeled Pajamas, pajamas::define, pajamas::design, pajamas::build, or pajamas::integrate. See Pajamas component lifecycle and the Pajamas issue tracker.
- Completing ToDo blocks in Pajamas for components or examples not fully specified.

Nice to do:

- Addressing issues labeled Seeking community contributions or other low-hanging usability

problems.

- Issues in future milestones (e.g., next release or Backlog).
- Popular issues with no milestone (based on comments or upvotes).
- Other issues labeled UX.
- Socializing design work outside GitLab (blog posts, social media, conferences).

Supporting Product Management

To aid Product Management in their prioritization efforts, we provide insights into UX needs, identifying both quick wins and larger strategic initiatives.

Engagement with Single Engineer Groups (SEGs)

The Engineering Department uses Single Engineer Groups (SEGs) to quickly develop "new market" initiatives. SEGs may request temporary design support for high-usage product areas.

Product Designers and managers should provide in-depth design critiques in issues and during MR reviews, collaborating with Incubation Engineering to ensure a great user experience.

Design reviews/critiques

- SEGs handle their design ideation. Product Designers are not responsible for solving customers problems or creating design assets for SEGs.
- SEGs can request feedback at any phase (early ideations, wireframes, high-fidelity mockups) by contacting the relevant Product Design Manager.
- Ideally, SEGs provide advance notice for design reviews for better planning and capacity management.

Reviewing SEG merge requests

- All MRs with user-facing changes require a designer review.
- Treat SEG MRs like community contributions, with the nearest area designer collaborating with the SEG.
- For new product areas without direct stage group impact, use GitLab Roulette to help assign a Product Designer to an MR for review. Manually spin the wheel to identify the designer.

UX Issue Weights

Issue weights are optional but useful for planning. They help Product Designers understand their capacity, evaluate the impact of time off, facilitate trade-off discussions with Product Managers, and identify teams needing more UX support.

Using UX Issue Weighting

1. Review upcoming issues and break down the work.
1. Assign an issue weight using the provided chart. Collaborate with the Product Manager for clarity if needed.
1. Record your issue weight using your team's preferred method.
 - List issues and weights in Google docs.
 - Create a linked issue for the UX work and add a weight (close it after completion).

- Use design-weight labels.
1. Communicate your capacity and total issue weights when planning iterations or milestones.
 1. Review weights post-milestone to improve future planning accuracy.

Additional Notes:

- Most UX issues, including research and Pajamas related issues can be weighted, if desired.
- Consider visual reviews, meetings, and other non-weighted responsibilities when determining capacity.
- Do not weight very small issues (less than 1).
- A weight of 8 or 13 indicates that the issue may be too large.
- Add weights to unplanned work during a milestone and discuss trade-offs with the Product Manager.

> Milestone Capacity Template:

>

- > - Total weights completed last milestone: 10
- > - Average capacity (average of weights completed last 3 months): 10
- > - Estimated capacity for current milestone: 7 (use your average capacity and subtract planned time off)
- > - Total current weights: 10

UX Weight Definitions

| Weight | Design Tasks | User Research Tasks |

| ----- | ----- | ----- |

| 1 | Mostly small UI changes leading to small incremental UX improvements. No users' workflow involved in these changes. Requirements are clear and there are no unanswered questions. *For example: A copy experiment or changing a button styling.* | Synthesizing previous research findings and generating recommendations based on them. |

| 2 | Simple UI or UX change where we understand all of the requirements but may need to find solutions to known questions/problems. These changes should blend in with an actual user workflow. *For example: Simplify Sign in / Register process the in trial flow.* | Running a first click test or other type of unmoderated research study |

| 3 | A well-understood change but the scope of work is bigger. Several pages are involved and/or we're starting to design/redesign small flows or connect existing flows between each other. Designers may conduct extensive background research (previous issues, support tickets, review past user research, review analytics, etc). Some unknown questions may arise during the work. *For example: Update the CustomersDot checkout page to allow subscription and billing information input, Experiment with adding a contact sales option in app.* | A moderated, narrowly scoped research study with specific questions to answer, such as a usability review of a single page |

| 5 | A complex change where input from group members is needed as early as possible. Spans across multiple pages, and we're working on medium-sized flows that potentially connect with another area of the product There are significant open questions that need to be answered. The product designer may need to do some research on their own or in collaboration with a researcher, but this isn't always the case. Possible research activities might be to find and/or validate a Job To Be Done, conduct user testing or card-sorting, or do a survey. *For example: UX Scorecard, How can we improve the dismiss action in upgrade moments.* | A research study evaluating the usability of an end-to-end flow or multiple related features, or

a usability study with a minor exploratory component |

| 8 | Complicated changes introducing a new user flow that connects with other large flows and may require input from other designers, product managers, or engineers from the same or another stage group. This is the largest flow design/redesign that we would take on in a single milestone. This requires research where the designer may or may not be working with a researcher to plan and conduct exploratory interviews or user testing sessions. *For example: Onboard new signs up through an onboarding issue board*. | An exploratory study investigating broad-based behaviors related to a single stage, including one or more distinct user groups |

| 13 | Highly significant changes impacting multiple user flows, a large new feature, and/or a complete redesign. This issue could significantly impact product strategy and would require critical input from others (the wider GitLab community, e-group, customers), and there are many unknowns. This necessitates research where the designer could team up with a researcher and other designers to gather input data, plan and conduct exploratory interviews, lead user testing sessions. It's unlikely we would commit to complete this issue in a milestone, and the preference would be to further clarify requirements and/or break it into smaller issues planned in several milestones. *For example: An improved free trial sign-up experience for GitLab.com SaaS users*. | An exploratory study investigating broad-based behaviors across multiple stages and multiple types of users, potentially involving team members from the different stages. |

SECTION: product/ux/product-designer/figma/_index.md

Figma & FigJam

Note: Unless otherwise stated, the information about Figma in this section also applies to FigJam.

Our primary design tool is Figma. As a product designer or product design manager in the UX department, you will have a Professional Figma license and access to the GitLab team in Figma. You don't need to do anything on your end, other than accept the invite sent to your GitLab email account during onboarding.

Anyone else in GitLab can access your files when you either share the file URL or invite them directly via email, but we ask that you only give them `·can view·` permissions. Anyone with `·can edit·` permissions is considered a paid seat and must have approval. A user with `·can view·` permission will still be able to comment on and inspect design files.

If you want to invite team members to collaborate on your FigJam boards, you can do this by starting an open session. This feature allows for team members with or without a FigJam license to have edit access as long as the board has been enabled for an open session and the link for the board is shared with them. Open sessions will only run for 24 hours at a time. If you want team members to be able to edit your files across multiple days, you will need to restart the open session each day.

If there are concerns about keeping information on your Figjam board SAFE, you can add password protection. That way only those with the password can edit the board when the open session is enabled.

GitLab has a public Figma profile where anyone can duplicate or remix files we have published. You can view our profile under the Community tab of the GitLab team section, or navigate to <https://www.figma.com/@GitLab>.

Do not create additional teams. An editor is billed for each team they are on, and we've only allocated resources for one team.

Figma has four levels of access, also called Permissions, for Professional teams. Permissions can be set at a team, project, and file level.

1. Editors - Only GitLab product designers and product design managers in the UX department are editors. Editors have ·can edit· access to projects and files, unless otherwise changed.
1. Viewers - Viewers on the GitLab team have ·can view· access to any project or file, unless otherwise changed.
1. Admins - An admin has access to Team Settings, and the Admin Dashboard, including billing.
1. Owners - Anyone who creates a team, project, or file will be the default owner of it.

[View complete permission details](#)

File backup

Figma's cloud storage contains all design files under our Organization account and only the Pajamas UI Kit files are published to our GitLab Product Design community page and available publicly. It's recommended that you back up any non-confidential design files you'd like to have a record of for future reference or portfolio use by saving a .fig file and storing via your preferred method.

SECTION: product/ux/product-designer/mr-reviews/_index.md

Requirements

Product Designers must review and approve MRs that include user-facing changes. According to the approval guidelines, user-facing changes encompass both visual changes (regardless of how minor) and changes to the rendered DOM affecting screen reader announcements.

MRs with backend changes that affect UX (e.g., performance, list sorting) do not require your review unless they are user-facing.

Stay aware of all MRs in your stage group and communicate with engineers about potential UX impacts. Use your judgment to decide which MRs to review, even those seemingly unrelated to UX.

We use GitLab Roulette to assign subject-matter experts for MRs. Learn more about MR reviews assignments.

Benefits

Familiarity with the product area and changes allows designers to:

- Efficiently set up specifications for local testing.
- Understand the rationale and context behind changes.
- Provide actionable feedback.
- Identify edge cases and bugs before merging code into production.

Collaborate closely with engineering peers throughout the product development lifecycle, from ideation to production, to strengthen product relationships and avoid a waterfall process.

How to assign MR reviews

Stage group MRs

GitLab Roulette assigns designers to stage group MRs. The design DRI acts as the reviewer for these MRs. If a stage group lacks a designer, MR reviews cannot be accommodated due to capacity issues.

Community contributions

Community-submitted MRs are assigned to the design DRI of the affected group. If the group has no designer, @pedroms or a member of UX Paper Cuts (@annabeldunstone, @seggenberger, @jmiocene, @clavimoniere) will review them. The GitLab Roulette suggests the correct designers automatically and generated a Slack message in the ux-community-contributions channel.

Single engineering group MRs

Single Engineer Group (SEG) MRs should be reviewed by the design DRI of the impacted group. If the group lacks a designer, MR reviews cannot be accommodated due to capacity issues.

Workload and response times

MR review requests are the top priority for Product Designers. Respond according to our review-response Service-Level Objective.

Balancing MR reviews with other tasks can be challenging. To avoid disruptions, block daily time to review MRs (for example, 30 minutes or 1 hour per day). If you struggle with reviews, manage expectations) with MR authors and reassess your capacity, considering any upcoming time off. If needed, work with your manager to reassign the MR.

Monitoring MR review workload

If overloaded with MRs, inform your manager immediately. Request assistance from another designer on your team or in the uxcoworking Slack channel. Product Design Managers should escalate and monitor these occurrences to determine if they indicate a broader trend.

Monitor MR review distribution using the GitLab Review Workload Dashboard and Product Design MR review volume.

Reviewing

Follow the Code Review guidelines (read in entirety). Exceptions to these guidelines are noted

below.

Understand the MR

Ensure the MR description includes:

- A thorough explanation of the changes.
- How to test the changes.
- Links to related issues.
- Before and After screenshots/videos (if appropriate).

For MRs with the ~"UX" label lacking a design DRI or proposed design, gather as much context as you can about the change. If unsure of the impacted product areas, involve other designers and your design manager.

Preview the MR

Always preview the MR in a live environment. Screenshots and videos are helpful but don't show everything (hover states, small screens, accessibility).

If you need help with previewing, see the help section.

Specific review requirements

Some MRs require additional setup:

- SaaS-only features: Run the GDK as a SaaS version. Simulate SaaS in the GDK.
- Paid features: Request a license via an Access Request, using the `GitLabTeamMemberLicenseRequest` template. Add the license to your instance.
 - You can also switch between CE and EE editions: [How to simulate a CE instance](#).
- Pipeline-related and Runner features: Create or enable a runner to run a pipeline. Create a runner in Gitpod or GDK.
- Compliance: To test audit event streaming using a stream destination URL. Generate a temporary destination with Pipedream.
- Fulfillment: Only Fulfillment Product Designers should review CustomersDot MRs.
 - Set up CustomersDot locally. If impractical, review screenshots and videos in the MR description or coordinate a demo with the engineer. For complex changes, keep the change behind a feature flag and review on staging post-merge.
- Geo: Install and configure two GDKs as Geo primary and secondary sites.
 - Simple installation.
 - Video and slide deck.
- Pipeline Execution: For compute minutes and shared runner usage related features, populate projects with historical compute minutes usage data. Set that up in under 7 minutes:
<details>
<summary>Populate compute minutes usage data</summary>

```
{{< youtube "ym-fU1U-anE" >}}
```

Checkout the branch in the MR and open rails console using `bin/rails console`.

1. Edit compute minutes

```
ruby
ApplicationSetting.current.update(sharedrunnersminutes: 400)
project = Project.find(20)
rootnamespace = project.rootnamespace
namespaceusage = Ci::Minutes::NamespaceMonthlyUsage.findorcreatecurrent(namespaceid:
rootnamespace.id)
```